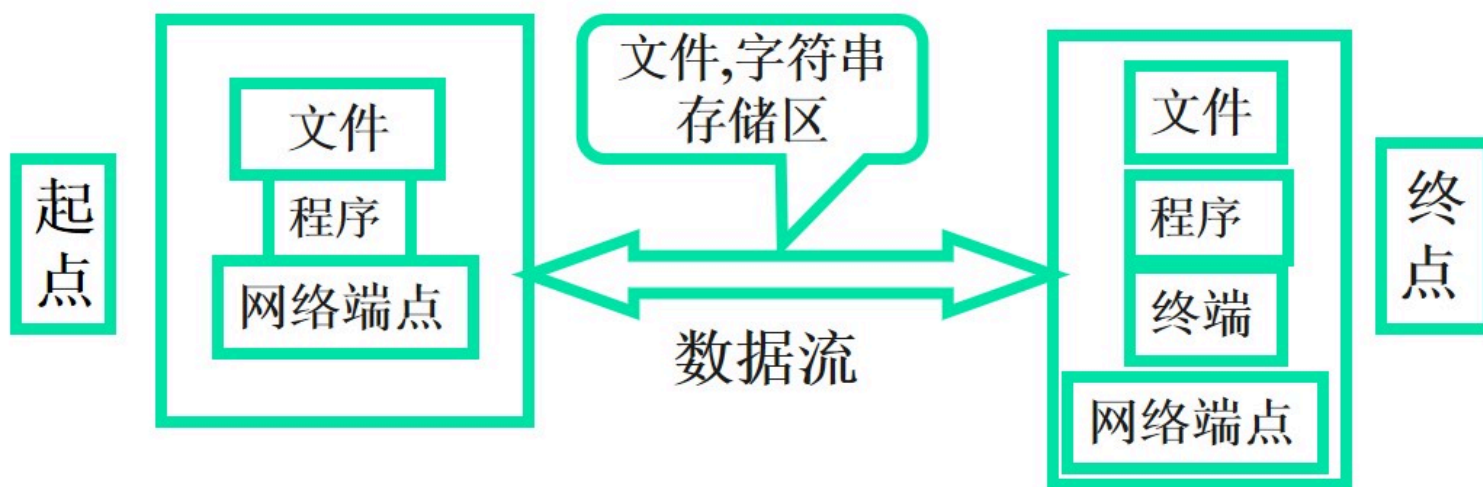


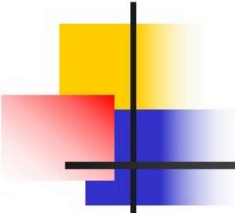
第十章 输入/输出处理

- I/O流概述
- 字节流
 - 字节流的基类
 - 文件流
 - 过滤流
 - 标准流
- 字符流
- 文件操作/随机访问文件
- 对象流
- 管道流
- 内存读写流
- 顺序输入流

1、I/O流概述

大部分程序都需要输入/输出处理，比如从键盘读取数据、向屏幕中输出数据、从文件中读或者向文件中写数据、在一个网络连接上进行读写操作等。在Java中，把这些不同类型的输入、输出源抽象为流（Stream），而其中输入或输出的数据则称为数据流（Data Stream），用统一的接口来表示，从而使程序设计简单明了。





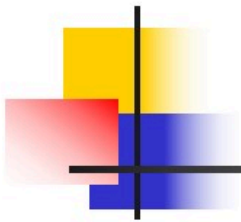
1、I/O流概述

流一般分为输入流（**Input Stream**）和输出流（**Output Stream**）两类，但这种划分并不是绝对的。比如一个文件，当向其中写数据时，它就是一个输出流；当从其中读取数据时，它就是一个输入流。当然，键盘只是一个输入流，而屏幕则只是一个输出流。

在Java开发环境中，主要是由包**java.io**中提供的一系列的类和接口来实现输入/输出处理。标准输入/输出处理则是由包**java.lang**中提供的类来处理的，但这些类又都是从包**java.io**中的类继承而来。

输入流：数据提供者，可从中读取数据出来

输出流：数据接收者，可往其中写数据



1、I/O流概述

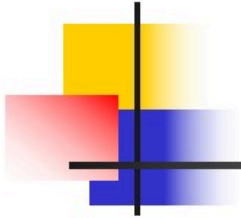
在JDK1.1之前，java.io包中的流只有普通的字节流（以byte为基本处理单位的流），这种流对于以16位的Unicode码表示的字符流处理很不方便。

从JDK1.1开始，java.io包中加入了专门用于字符流处理的类（以Reader和Writer为基础派生的一系列类）。

另外，为了使对象的状态能够方便地永久保存下来，JDK1.1以后的java.io包中提供了以字节流为基础的用于对象的永久化保存状态的机制——对象流（通过实现ObjectInput和ObjectOutput接口）。

1、I/O流概述

——字节流



■ *InputStream*

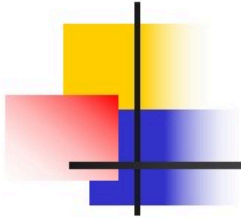
- ByteArrayInputStream
- FileInputStream
- *FilterInputStream*
 - BufferedInputStream
 - DataInputStream
 - *LineNumberInputStream*
 - PushbackInputStream
- ObjectInputStream
- PipedInputStream
- SequenceInputStream
- *StringBufferInputStream*

■ *OutputStream*

- ByteArrayOutputStream
- FileOutputStream
- *FilterOutputStream*
 - BufferedOutputStream
 - DataOutputStream
 - PrintStream
- ObjectOutputStream
- PipedOutputStream

1、I/O流概述

——字符流



■ *Reader*

- `BufferedReader`
 - `LineNumberReader`
- `CharArrayReader`
- *`FilterReader`*
 - `PushbackReader`
- `InputStreamReader`
 - `FileReader`
- `PipedReader`
- `StringReader`

■ *Writer*

- `BufferedWriter`
- `CharArrayWriter`
- *`FilterWriter`*
- `OutputStreamWriter`
 - `FileWriter`
- `PipedWriter`
- `StringWriter`
- `PrintWriter`

1、I/O流概述

——I/O接口 与 文件操作

- DataInput
 - ObjectInput
- DataOutput
 - ObjectOutput
- FileFilter
- FilenameFilter
- ObjectInputValidation
- ObjectStreamConstants
- Serializable
 - Externalizable
- File
- FileDescriptor
- RandomAccessFile
- java.awt.FileDialog

java.io类详解

2、字节流

——基类：InputStream

Java中每一种字节流的基本功能依赖于基本类InputStream和OutputStream，它们是抽象类，不能直接使用。

属于InputStream类的方法有：

- read(): 从流中读入数据
- skip(): 跳过流中若干字节数
- available(): 返回流中可用字节数
- mark(): 在流中标记一个位置
- reset(): 返回标记过的位置
- markSupport(): 是否支持标记和复位操作
- close(): 关闭流

2、字节流

——基类: `InputStream`

在`InputStream`类中，方法`read()`提供了三种从流中读数据的方法：

- `int read()`: 从输入流中读一个字节，形成一个0~255之间的整数返回（是一个抽象方法）。
- `int read(byte b[])`: 读多个字节到数组中，填满整个数组。
- `int read(byte b[], int off, int len)`: 从输入流中读取长度为`len`的数据，写入数组`b`中从索引`off`开始的位置，并返回读取得字节数。

对于这三个方法，若返回-1，表明流结束，否则，返回实际读取的字节数。

2、字节流

——基类: **OutputStream**

属于OutputStream类的方法有:

- `write(int b)`: 将一个整数输出到流中（只输出低位字节, 为抽象方法）
- `write(byte b[])`: 将字节数组中的数据输出到流中
- `write(byte b[], int off, int len)`: 将数组b中从off指定的位置开始, 长度为len的数据输出到流中
- `flush()`: 刷空输出流, 并将缓冲区中的数据强制送出
- `close()`: 关闭流

Writer的方法:

`write(String s), write(String s, int off, int len)`



2、字节流

——基类：例子

- 把输入流中的所有内容复制到输出流中

```
public void copy(InputStream in, OutputStream out)
    throws IOException {
    byte[] buf = new byte[4096];
    int len = in.read(buf);
    while (len != -1) {
        out.write(buf, 0, len);
        len = in.read(buf);
    }
}
```

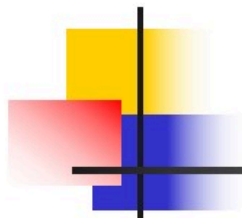

2、字节流

——各种字节流

- 文件流
- 过滤流：缓冲流、数据流、其他过滤流
- 标准流
- 对象流
- 管道流
- 内存流
- 顺序输入流

2、字节流

——文件流



在I/O处理中，最常见的就是对文件的操作。`java.io`包中所提供的文件操作类包括：

`FileInputStream:`
`FileOutputStream:` } 用于读本地文件系统中的文件

`File:`
`FileDescriptor:` } 描述本地文件系统中的文件或目录

`FilenameFilter:` 接口，主要用于实现文件名查找模式的匹配。

`RandomAccessFile:` 提供对本地文件系统中文件的随机访问支持。

2、字节流

——文件流： `FileInputStream/ FileOutputStream`

- ❑ `FileInputStream`类用来打开一个输入文件，若要打开的文件不存在，则会产生例外`FileNotFoundException`，这是一个非运行时例外，必须捕获或声明抛弃；
- ❑ `FileOutputStream`类用来打开一个输出文件，若要打开的文件不存在，则会创建一个新的文件，否则原文件的内容会被新写入的内容所覆盖。
- ❑ 在进行文件的读/写操作时，会产生非运行时例外`IOException`，必须捕获或声明抛弃（其它的输入/输出流处理时也同样需要进行输入/输出例外处理）。

2、字节流

——文件流: `FileInputStream/ FileOutputStream`

- 文件流的构造方法:
 - `FileInputStream(File f)`
 - 打开一个以*f*描述的文件作为输入。
 - `FileInputStream(String name)`
 - 打开一个文件路径名为*name*的文件作为输入。
 - `FileOutputStream(File f)`
 - 创建一个以*f*描述的文件作为输出，文件如果已经存在，则其内容被清空。
 - `FileOutputStream(String name)`
 - 创建一个文件路径名为*name*的文件作为输出，文件如果已经存在，则其内容被清空。
 - `FileOutputStream(String name, boolean append)`
 - 创建一个文件路径名为*name*的文件作为输出，文件如果已经存在，则在该输出上输出的内容被接到原有内容之后。

2、字节流

——文件流: `FileInputStream/ FileOutputStream`

```
File f1 = new File("file1.txt");
```

```
File f2 = new File("file2.txt");
```

```
FileInputStream in=new FileInputStream(f1);
```

```
FileOutputStream out=new FileOutputStream(f2);
```

```
FileOutputStream out=new FileOutputStream("file3.txt");
```

输入流的参数是用于指定输入的文件名，输出流的参数则是用于指定输出的文件名。



2、字节流——文件流：例子1

将一个文件复制到另一个文件中（覆盖）

```
import java.io.*;
class Filestream
{
    public static void main(String args[])
    {
        try
        {
            File inFile=new File("file1.txt");
            File outFile=new File("file2.txt");
            FileInputStream fis=new FileInputStream(inFile);
            FileOutputStream fos=new FileOutputStream(outFile);
            int c;
            while((c=fis.read())!=-1) fos.write(c);
            fis.close();
            fos.close();
        }catch(FileNotFoundException e) {
            System.out.println("FileStreamsTest: "+e);
        }catch(IOException e) {
            System.err.println("FileStreamsTest: "+e);
        }
    }
}
```

2、字节流

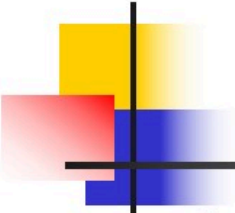
——文件流：例子2

- 把一个文件的内容加到另一个文件后

```
public void cat(String fsrc, String fdest) {  
    try {  
        InputStream in = new FileInputStream(fsrc);  
        OutputStream out =  
            new FileOutputStream(fdest, true);  
        copy(in, out);  
        out.close();  
        in.close();  
    }  
    catch (IOException ex) {  
        System.err.println(ex);  
    }  
}
```


2、字节流

——过滤流



java.io中提供类FilterInputStream和FilterOutputStream分别对其他输入/输出流进行特殊处理，它们在读/写数据的同时可以对数据进行特殊处理。另外还提供了同步机制，使得某一时刻只有一个线程可以访问一个输入/输出流。

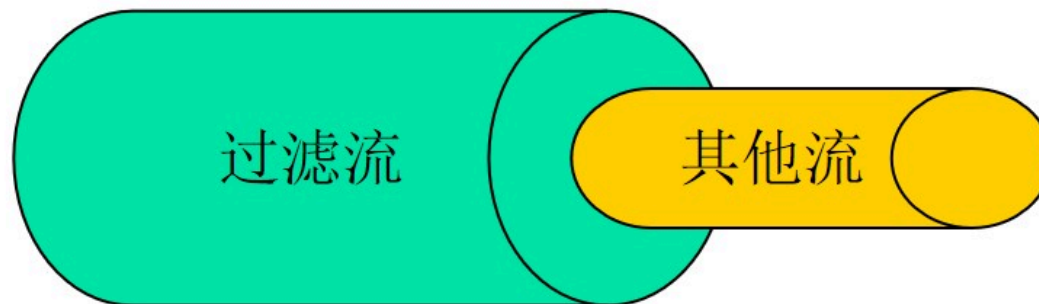
类FilterInputStream和FilterOutputStream分别重写了父类InputStream和OutputStream的所有方法，同时，它们的子类也应该重写它们的方法以满足特定的需要。

要使用过滤流，首先必须把它连接到某个输入/输出流上，通常在构造方法的参数中指定所要连接的流：

```
protected FilterInputStream(InputStream in);  
protected FilterOutputStream(OutputStream out);
```

2、字节流

——过滤流



2、字节流

——过滤流：缓冲流

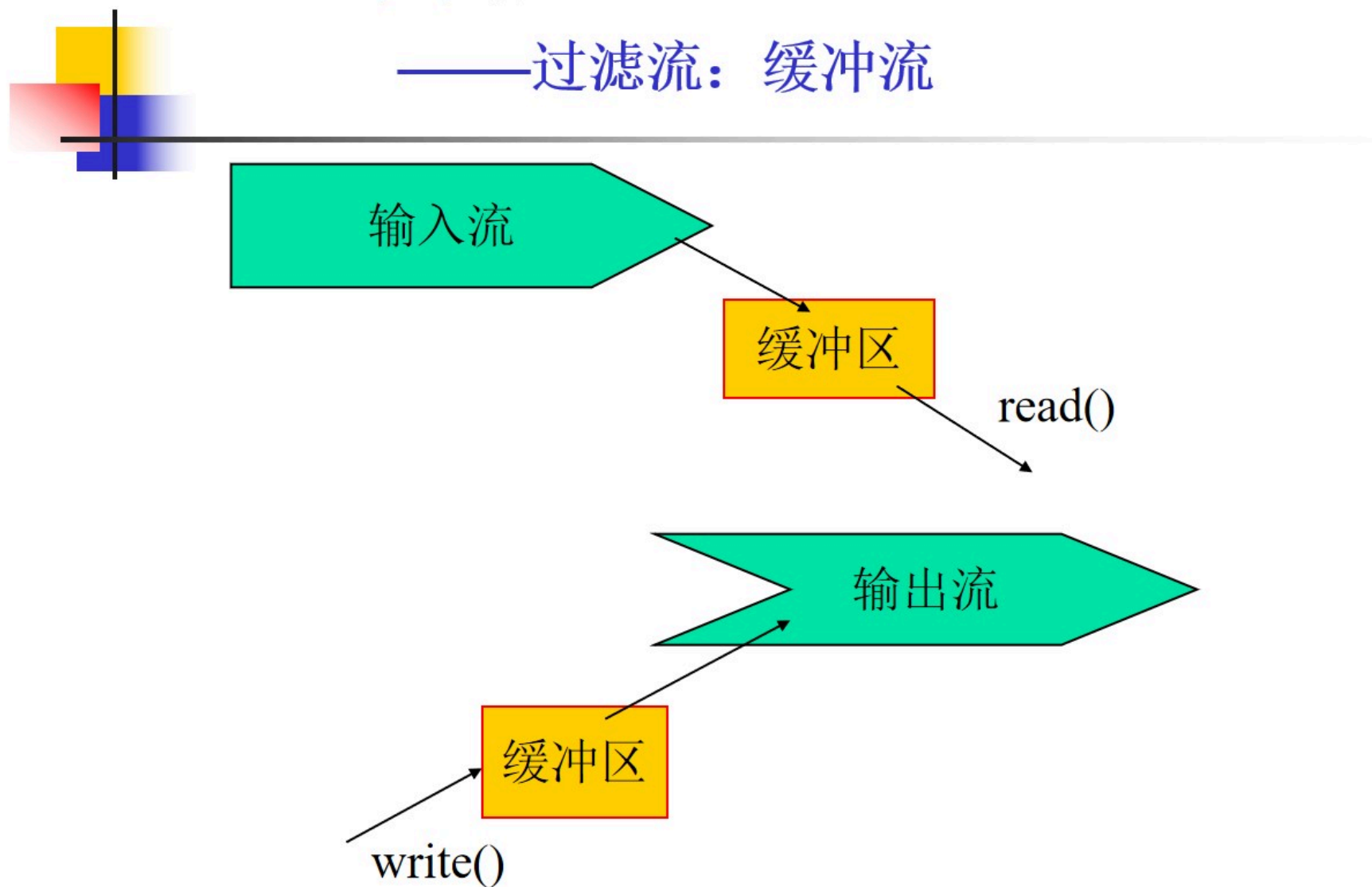
类BufferedInputStream和BufferedOutputStream实现了带缓冲的过滤流，它提供了缓冲机制，把任意的I/O流“捆绑”到缓冲流上，可以提高该I/O流的读取效率。

在初始化时，除了要指定所连接的I/O流之外，还可以指定缓冲区的大小。缺省时是用32字节大小的缓冲区；最优的缓冲区大小常依赖于主机操作系统、可使用的内存空间以及机器的配置等；一般缓冲区的大小为内存页或磁盘块等的整数倍，如8912字节或更小。

- BufferedInputStream(InputStream in[, int size])
- BufferedOutputStream(OutputStream out[, int size])

2、字节流

——过滤流：缓冲流



——过滤流：缓冲流

将缓冲流与文件流相接:

```
FileInputStream in = new FileInputStream("file1.txt");
```

```
FileOutputStream out = new FileOutputStream ("file2.txt");
```

```
BufferedInputStream bin = new BufferedInputStream(in,256)
```

```
BufferedOutputStream bout = new BufferedOutputStream(out,256);
```

```
int len;
```

```
byte bArray[]=new byte[256];
```

len=bin.read(bArray); //len中得到的是实际读取的长度, bArray中得到的是数据

file1.txt $\xRightarrow{\text{输入流}}$ 输入缓冲流 $\rightarrow$ 输出缓冲流 $\xRightarrow{\text{输出流}}$ file2.txt

文件→文件流→缓冲流

2、字节流

——过滤流：缓冲流

- 对于`BufferedOutputStream`，只有缓冲区满时，才会将数据真正送到输出流，但可以使用`flush()`方法人为地将尚未填满的缓冲区中的数据送出。

```
public void copy(InputStream in, OutputStream out)
    throws IOException {
    out = new BufferedOutputStream(out, 4096);
    byte[] buf = new byte[4096];
    int len = in.read(buf);
    while (len != -1) {
        out.write(buf, 0, len);
        len = in.read(buf);
    }
    out.flush(); //最后一次读取的数据可能不到4096字节
}
```

2、字节流

——过滤流：缓冲流（例子）

- 使用缓冲流支持的 **mark**和**reset**机制

```
public String readLine(  
    BufferedInputStream in)  
    throws IOException {  
    StringBuffer sb =  
        new StringBuffer();  
    int c = in.read();  
    .....  
    return sb.toString();  
}
```

```
while (c != -1) {  
    if (c == '\n')  
        break;  
    if (c == '\r') {  
        in.mark(1);  
        if (in.read() != '\n')  
            in.reset();  
        break;  
    }  
    sb.append((char) c);  
    c = in.read();  
}
```

2、字节流

——过滤流：数据流

接口DataInput和DataOutput，设计了一种较为高级的数据输入输出方式：除了可处理字节和字节数组外，还可以处理int、float、boolean等基本数据类型，这些数据在文件中的表示方式和它们在内存中的一样，无须转换，如read(), readInt(), readByte() ...; write(), writeChar(), writeBoolean()...。此外，还可以用readLine()方法读取一行信息。

2、字节流

——过滤流：数据流（**DataInput**）

- `boolean readBoolean()`
- `byte readByte()`
- `short readShort()`
- `char readChar()`
- `int readInt()`
- `long readLong()`
- `double readDouble()`
- `float readFloat()`
- `int readUnsignedByte()`
- `int readUnsignedShort()`

2、字节流

——过滤流：数据流（DataInput）

- `void readFully(byte[] b)`
 - 读满字节数组，不同于`InputStream.read`
- `void readFully(byte[] b, int off, int len)`
 - 读满指定长度，不同于`InputStream.read`
- `int skipBytes(int n)`
 - 与`InputStream.skip`等价
- `String readUTF()`
 - 按UTF-8形式从输入中读取字符串
- `String readLine()`
 - 按回车(`\r`)换行(`\n`)为分割符读取一行字符串
 - 不完全支持UNICODE

2、字节流

——过滤流：数据流（DataOutput）

- void writeBoolean(boolean v)
- void writeByte(int v)
- void writeShort(int v)
- void writeChar(int v)
- void writeInt(int v)
- void writeLong(long v)
- void writeFloat(float v)
- void writeDouble(double v)

2、字节流

——过滤流：数据流（DataOutput）

- `void write(byte[] b)`
 - 与`OutputStream.write`同义
- `void write(byte[] b, int off, int len)`
 - 与`OutputStream.write`同义
- `void write(int b)`
 - 与`OutputStream.write`同义
- `void writeBytes(String s)`
 - 只输出每个字符的低8位；不完全支持UNICODE。
- `void writeChars(String s)`
 - 每个字符在输出中都占两个字节。

2、字节流

——过滤流：数据流

- 数据流类 **DataInputStream**和**DataOutputStream**的处理对象除了是字节或字节数组外，还可以实现对文件的不同数据类型的读写：
 - 分别实现了**DataInput**和**DataOutput**接口
 - 在提供了字节流的读写手段的同时，
 - 以统一的通用的形式向输入流中写入**boolean**, **int**, **long**, **double**等基本数据类型，并可以在次把基本数据类型的值读取回来。
 - 提供了字符串读写的手段。

2、字节流

——过滤流：数据流

数据流可以连接一个已经建立好的数据对象,例如网络的连结、文件等。数据流可通过如下方式建立:

```
FileInputStream fis = new FileInputStream("file1.txt");  
FileOutputStream fos = new FileOutputStream("file2.txt");  
DataInputStream dis = new DataInputStream(fis);  
DataOutputStream dos = new DataOutputStream(fos);
```

```
class DataStream
{
    public static void main(String args[]) throws IOException
    {
        FileOutputStream fos = new FileOutputStream("a.txt");
        DataOutputStream dos = new DataOutputStream (fos);
        try
        {
            dos.writeBoolean(true);
            dos.writeByte((byte)123);
            dos.writeChar('J');
            dos.writeDouble(3.141592654);
            dos.writeFloat(2.7182f);
            dos.writeInt(1234567890);
            dos.writeLong(998877665544332211L);
            dos.writeShort((short)11223);
        }
        finally
        {
            dos.close();
        }
    }
}
```



```
FileInputStream fis = new FileInputStream("a.txt");
DataInputStream dis = new DataInputStream(fis);
try
{
    System.out.println("\t "+dis.readBoolean());
    System.out.println("\t "+dis.readByte());
    System.out.println("\t "+dis.readChar());
    System.out.println("\t "+dis.readDouble());
    System.out.println("\t "+dis.readFloat());
    System.out.println("\t "+dis.readInt());
    System.out.println("\t "+dis.readLong());
    System.out.println("\t "+dis.readShort());
}
finally
{
    dis.close();
}
} //main()
} //class DataStream
```

过滤流：
数据流（例子一）

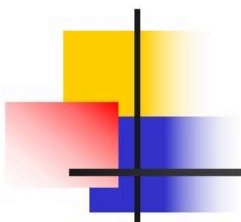
2、字节流

——过滤流：数据流（例子2）

//利用方法**readLine()**计算一个输入流中的字符数和行数

//（适合于文本文件）

```
int DataLine(InputStream in)
{
    DataInputStream data = new DataInputStream(in);
    String currentLine;
    int lineCount=0;
    int charCount=0;
    while((currentLine=data.readLine())!=null)
    {
        ++lineCount;
        charCount += currentLine.length();
    }
    return (charCount/(float)lineCount);
}
```



字节流进行读写时不支持中文的,会出现乱码.而且由于采用的是字节流,它会把你上面的**20,180**转换为对应的值存入流中.如果把**20**改为**70**(对应的**ASCII**码值为**F**),就能看到文件中被存入了**F**.

至于读的时候,只有它自己才知道它写入了什么.

2、字节流

——过滤流：其他过滤流

LineNumberInputStream：主要用于对文本文件的处理，提供了行号控制功能。

- 已经被LineNumberReader取代

PushBackInputStream：在编译程序的词法分析阶段，经常要超前读入一个字节以界定当前词的属性，然后再将该字节退回（因为下面的处理可能还会用到该字节）。PushBackInputStream就提供了这样的能力，它提供了一个方法将刚刚读入的字节退回到输入流中去。

PrintStream：其作用是将Java语言中的不同类型的数据以字符表示形式输出到相应的输出流中去。

- 不产生异常。可自动flush。通过checkError()检查错误。

2、字节流

——标准流

语言包java.lang中的System类管理标准输入/输出流和错误流。

System.in，从InputStream中继承而来，用于从标准输入设备中获取输入数据(通常是键盘)。

System.out，从PrintStream中继承而来，把输出送到缺省的显示设备(通常是显示器)。

System.err，也是从PrintStream中继承而来，把错误信息送到缺省的显示设备(通常是显示器)。

每当main方法被执行时，就自动生成上述三个对象。


```
public class ReadFromKB
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        try
```

```
        {
```

```
            byte bArray[]=new byte[128];
```

```
            String str;
```

```
            System.out.println("Enter something Using Keyborad:");
```

```
            System.in.read(bArray);
```

```
            str = new String(bArray, 0);
```

```
            System.out.print("You entered:");
```

```
            System.out.println(str);
```

```
        }
```

```
        catch(IOException ioe)
```

```
        {
```

```
            System.out.println(ioe.toString());
```

```
        }
```

```
    }
```

```
}
```

过滤流：
标准流（例子）
从键盘中获取数据

2、字节流

——对象流

- 对象的持续性（Persistence）

能够纪录自己的状态以便将来再生的能力，叫对象的持续性。

- 对象的串行化（Serialization）

对象通过写出描述自己状态的数值来记录自己的过程叫串行化。串行化的主要任务是写出对象实例变量的数值，如果变量是另一个对象的引用，则引用的对象也要串行化。这个过程是递归的。

- 对象流

能够输入输出对象的流称为对象流。

可以将对象串行化后通过对象输入输出流写入文件或传送到其它地方。

2、字节流

——对象流

在Java中，允许可串行化的对象在通过对象流进行传输。只有实现Serializable接口的类才能被串行化，Serializable接口中没有任何方法，当一个类声明实现Serializable接口时，只是表明该类加入对象串行化协议。

```
public class Student implements Serializable
{
    int id;
    String name;
    int age;
    String department;
    public Student(int id, String name, int age, String department)
    {
        this.id=id;
        this.name=name;
        this.age=age;
        this.department =department;
    }
}
```


2、字节流

——对象流

要串行化一个对象，必须与一定的对象输出/输入流联系起来，通过对象输出流将对象状态保存下来(将对象保存到文件中，或者通过网络传送到其他地方)，再通过对象输入流将对象状态恢复。

类ObjectOutputStream和ObjectInputStream分别继承了接口ObjectOutput和ObjectInput，将数据流功能扩展到可以读写对象，前者用writeObject()方法可以直接将对象保存到输出流中，而后者用readObject()方法可以直接从输入流中读取一个对象。

2、字节流

——对象流

从某种意义来看，对象流与数据流是相类似的，也具有过滤流的特性。利用对象流来输入/输出对象时，也不能单独使用，需要与其他的流连接起来。

- ❑ 对象是可串行化的
- ❑ 使用对象流

```
public class Objectser
{
    public static void main(String args[])
    {
        Student stu=new Student(981036, "Li Ming", 16, "CSD");
        try
        {
            FileOutputStream fo = new FileOutputStream("data.ser");
            ObjectOutputStream so = new ObjectOutputStream(fo);
            so.writeObject(stu);
            so.close();
        }
        catch(Exception e)
        {
            System.out.println(e) ;
        }
    }
}
```

对象流：
例子（对象的存储）

对象流：
例子（对象的存储）

```
public class ObjectRecov
{
    public static void main(String args[])
    {
        Student stu;
        try
        {
            FileInputStream fi = new FileInputStream("data.ser");
            ObjectInputStream si = new ObjectInputStream(fi);
            stu = (Student)si.readObject();
            si.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
        System.out.println("ID: "+stu.id+"name:"+
            stu.name+"age:"+stu.age+"dept.:"+stu.department);
    }
}
```

2、字节流

——对象流

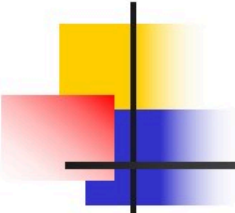
定制对象的串行化：当一个对象串行化时，如果希望该对象的某些属性不被保存，可以通过在类定义中重写readObject()和WriteObject()方法来实现。

定制对象的串行化

```
public class Student implements Serializable
{
    int id;
    String name;
    int age;
    String department;
    public Student(int id, String name, int age, String department)
    {
        this.id=id;
        this.name=name;
        this.age=age;
        this.department =departmernt;
    }
    private void writeObject(ObjectOutputStream out) throws IOException
    {
        out.writeInt(id);
        ... // out.defaultWriteObject()
    }
    private void readObject(ObjectInputStream in) throws IOException
    {
        id = in.readInt();
        ... // in.defaultReadObject()
    }
}
```

2、字节流

——对象流



串行化只能保存对象的非静态成员变量（实例变量），而不能保存任何成员方法和静态成员变量，并且保存的只是变量的值，对于变量的任何修饰符都不能保存。

对于某些类型的对象，其状态是瞬时的，这样的对象是无法保存其状态的，如Thread对象或流对象。对于这样的成员变量，必须用transient关键字标明，否则编译器将报错。任何用transient关键字标明的成员变量，都不会被保存。

另外，串行化可能涉及将对象存放到磁盘上或在网络上发送数据，这时会产生安全问题。对于一些需要保密的数据，不应保存在永久介质中（或者不应简单地不加处理地保存下来），为了保证安全，应在这些变量前加上transient关键字。

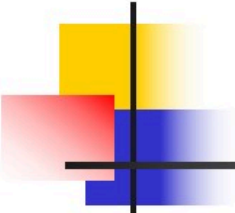
2、字节流

——管道流

管道用来把一个程序、线程和代码块的输出连接到另一个程序、线程和代码块的输入。java.io中提供了类PipedInputStream和PipedOutputStream作为管道的输入/输出流。

管道输入流作为一个通信管道的接收端，管道输出流则作为发送端。管道流必须是输入输出流并用，即在使用管道前，两者必须进行连接。





字节流

——管道流

管道输入/输出流可以用两种方式进行连接:

1) 在构造方法中进行连接

```
PipedInputStream(PipedOutputStream pos);
```

```
PipedOutputStream(PipedInputStream pis);
```

2) 通过各自的connect()方法连接

在类PipedInputStream中,

```
connect(PipedOutputStream pos);
```

在类PipedOutputStream中,

```
connect(PipedInputStream pis);
```


将数据从输出管道进，从输入管道出

```
class Pipedstream
{
    public static void main(String args[]) throws IOException
    {
        byte aByteData1 = 123, aByteData2 = 111;
        PipedInputStream pis = new PipedInputStream();
        PipedOutputStream pos = new PipedOutputStream(pis);
        System.out.println("PipedInputStream");
        try
        {
            pos.write(aByteData1);
            pos.write(aByteData2);
            System.out.println((byte)pis.read());
            System.out.println((byte)pis.read());
        }
        finally
        {
            pis.close();
            pos.close();
        }
    }
}
```

管道流：
例子

2、字节流

——内存读写流

- 为了支持在内存上的I/O, `java.io`中提供了类
 - `ByteArrayInputStream`
 - `ByteArrayOutputStream`
 - `StringBufferInputStream`
- `ByteArrayInputStream`可以从指定的字节数组中读取数据。
- `ByteArrayOutputStream`中提供了缓冲区可以存放数据（缓冲区大小可以在构造方法中设定，缺省为32），可以用`write()`方法向其中写入数据，然后用`toByteArray()`方法将缓冲区中的有效字节写到字节数组中去。`size()`方法可以知道写入的字节数；`reset()`可以丢弃所有内容。
- `StringBufferInputStream`与`ByteArrayInputStream`相类似，不同点在于它是从字符缓冲区`StringBuffer`中读取16位的Unicode数据，而不是8位的字节数据。（已被**`StringReader`**取代）

2、字节流

——内存读写流

- ByteArrayInputStream
 - ByteArrayInputStream(byte[] buf)
 - ByteArrayInputStream(byte[] buf, int offset, int length)
- ByteArrayOutputStream
 - void reset() : 重写内容
 - int size() : 返回写入的字节数
 - byte[] toByteArray() : 以新分配的字节数组形式返回写入的内容
 - String toString() : 以缺省字符编码方式把内容编程字符串返回
 - String toString(String enc) : 以指定字符编码方式返回字符串
 - void writeTo(OutputStream out) : 把内容写到另一个输出流中

2、字节流

——顺序输入流

`java.io`中提供了类**`SequenceInputStream`**，使应用程序可以将几个输入流顺序连接起来，让程序员看起来就像是一个比较长的流一样。顺序输入流提供了将多个不同的输入流统一为一个输入流的功能，这使得程序可能变得更加简洁。如：

```
FileInputStream f1,f2;  
String s;  
f1 = new FileInputStream("file1.txt");  
f2 = new FileInputStream("file2.txt");  
SequenceInputStream fs = new SequenceInputStream(f1, f2);  
DataInputStream ds = new DataInputStream(fs);  
while( (s = ds.readLine()) != null )  
    System.out.println(s);
```


3、字符流

前面说过，在JDK1.1之前，`java.io`包中的流只有普通的字节流（以`byte`为基本处理单位的流），这种流对于以16位的Unicode码表示的字符流处理很不方便。从JDK1.1开始，`java.io`包中加入了专门用于字符流处理的类，它们是以`Reader`和`Writer`为基础派生的一系列类。

同类`InputStream`和`OutputStream`一样，`Reader`和  `Writer`也是抽象类，只提供了一系列用于字符流处理的接口。它们的方法与类`InputStream`和`OutputStream`类似，只不过其中的参数换成字符或字符数组。

字节流中类`DataInputStream`的`readLine`方法，可以以字节形式读入，以Unicode形式输出（`String readLine()`）。



3、字符流

——基类: Reader

- void close()
- void mark(int readAheadLimit)
- boolean markSupported() :
- int read()
- int read(char[] cbuf)
- int read(char[] cbuf, int off, int len)
- boolean ready()
- void reset()
- long skip(long n)

3、字符流

——基类: **Writer**

- `void close()`
- `void flush()`
- `void write(char[] cbuf)`
- `void write(char[] cbuf, int off, int len)`
- `void write(int c)`
- `void write(String str)`
- `void write(String str, int off, int len)`

3、字符流

——InputStreamReader和OutputStreamWriter

InputStreamReader和OutputStreamWriter是java.io包中用于处理字符流的最基本的类，用来在字节流和字符流之间作为中介：从字节输入流读入字节，并按编码规范转换为字符；往字节输出流写字符时先将字符按编码规范转换为字节。使用这两者进行字符处理时，在构造方法中应指定一定的平台规范，以便把以字节方式表示的流转换为特定平台上的字符表示。

```
InputStreamReader(InputStream in);           //缺省规范
InputStreamReader(InputStream in, String enc); //指定规范enc

OutputStreamWriter(OutputStream out);        //缺省规范
OutputStreamWriter(OutputStream out, String enc); //指定规范enc
```


3、字符流

——InputStreamReader和OutputStreamWriter

如果读取的字符流不是来自本地时（比如网上某处与本地编码方式不同的机器），那么在构造字符输入流时就不能简单地使用缺省编码规范，而应该指定一种统一的编码规范“**ISO 8859_1**”，这是一种映射到**ASCII**码的编码方式，能够在不同平台之间正确转换字符。

```
InputStreamReader ir = new InputStreamReader( is, "8859_1" );
```

3、字符流

——缓存流: **BufferedReader**和**BufferedWriter**

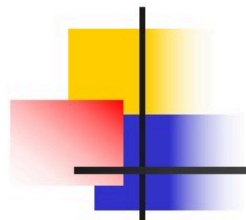
同样的，为了提高字符流处理的效率，**java.io**中也提供了缓冲流**BufferedReader**和**BufferedWriter**。其构造方法与**BufferedInputStream**和**BufferedOutputStream**相类似。另外，除了**read()**和**write()**方法外，它还提供了整行字符处理方法：

- **public String readLine():** **BufferedReader**的方法，从输入流中读取一行字符，行结束标志为 ‘\n’、 ‘\r’或两者一起。
- **public void newLine():** **BufferedWriter**的方法，向输出流中写入一个行结束标志，它不是简单的换行符 ‘\n’或 ‘\r’，而是系统定义的行隔离标志（line separator）。

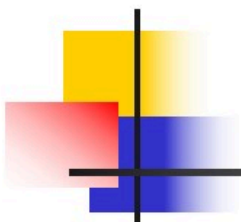
```
class FileToUnicode
{
    public static void main(String args[])
    {
        try
        {
            FileInputStream fis = new FileInputStream("file1.txt");
            InputStreamReader dis = new InputStreamReader(fis);
            // InputStreamReader dis = new InputStreamReader(System.in);
            BufferedReader reader = new BufferedReader(dis);
            String s;
            while( (s = reader.readLine()) != null )
            {
                System.out.println("read: "+s);
            }
            dis.close();
        } catch(IOException e)
        {
            System.out.println(e);
        }
    }
} //main()
} //class
```


3、字符流

——其它字符流



CharArrayReader CharArrayWriter	}	对字符数组进行处理
FileReader FileWriter	}	对文本文件进行处理
StringReader StringWriter	}	对字符串进行处理
FilterReader FilterWriter	}	过滤字符流
PipedReader PipedWriter	}	管道字符流
LineNumberReader: PrintWriter:		行处理字符流 打印字符流

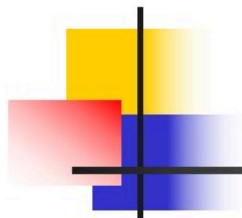


4、文件操作/随机访问文件

- **File:** 以文件路径名的形式代表一个文件
- **FileDescriptor:** 代表一个[打开文件](#)的文件描述
- **FileFilter & FilenameFilter:** 用于列出满足条件的文件
 - `File.list(FilenameFilter fnf)`
 - `File.listFiles(FileFilter ff)`
 - `FileDialog.setFilenameFilter(FilenameFilter fnf)`
- **FileInputStream & FileReader:** 顺序读文件
- **FileOutputStream & FileWriter:** 顺序写文件
- **RandomAccessFile:** 提供对文件的随机访问支持。

4、文件操作/随机访问文件

——文件操作: File



- File(String pathname)
 - `File f=new File("c:\data\temp.dat");`
 - `File f=new File("data\ temp.dat");`
 - `File f=new File("temp.dat");`
- File(String parent, String child)
 - `File f=new File("c:\data" , "temp.dat");`
 - `File f=new File("data " , " temp.dat");`
- File(File parent, String child)
 - `File f=new File(new File("c:\data") , "temp.dat");`
 - `File f=new File(new File("data ") , " temp.dat");`

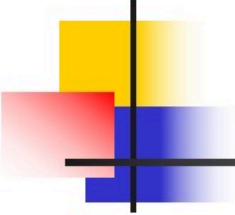
4、文件操作/随机访问文件

——文件操作：File

- boolean canRead()
- boolean canWrite()
- boolean setReadOnly()
- boolean exists()
- boolean isDirectory()
- boolean isFile()
- boolean isHidden()
- long lastModified()
- boolean setLastModified(long time)
- long length()

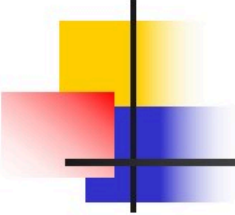
4、文件操作/随机访问文件

——文件操作：File

- 
- String[] list()
 - String[] list(FilenameFilter filter)
 - File[] listFiles()
 - File[] listFiles(FileFilter filter)
 - File[] listFiles(FilenameFilter filter)
 - static File[] listRoots()
 - boolean mkdir()
 - boolean mkdirs()
- （粉色的方法在JDK1.2之后才支持）

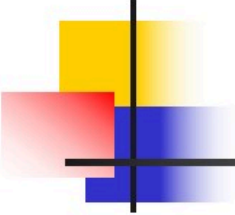
4、文件操作/随机访问文件

——文件操作：File

- 
- `boolean createNewFile()`
 - `static File createTempFile(String prefix, String suffix)`
 - `static File createTempFile(String prefix, String suffix, File directory)`
 - `boolean delete()`
 - `void deleteOnExit()`
 - `boolean renameTo(File dest)`

4、文件操作/随机访问文件

——文件操作：File

- 
- String getName()
 - File getParentFile()
 - String getParent()
 - String getPath()
 - boolean isAbsolute()
 - File getAbsolutePath()
 - String getAbsolutePath()
 - File getCanonicalFile()
 - String getCanonicalPath()

4、文件操作/随机访问文件

——文件操作: **FileDescriptor**

- FileInputStream & FileOutputStream & RandomAccessFile
 - FileDescriptor getFD()
- 通过FileDescriptor构造输入输出流
 - FileInputStream(FileDescriptor fdObj)
 - FileOutputStream(FileDescriptor fdObj)
 - FileReader(FileDescriptor fd)
 - FileWriter(FileDescriptor fd)
- 例如:
 - `FileInputStream fin = new FileInputStream("file.txt");`
 - `FileReader fr = new FileReader(fin.getFD());`

4、文件操作/随机访问文件

——随机访问文件（ **RandomAccessFile** ）

对于FileInputStream/FileOutputStream、FileReader/FileWriter来说，它们的实例都是顺序访问流，即只能进行顺序读/写。而类RandomAccessFile则允许对文件内容同时完成读和写操作，它直接继承object，并且同时实现了接口DataInput和DataOutput，提供了支持随机文件操作的方法：

- readXXX()或writeXXX()：

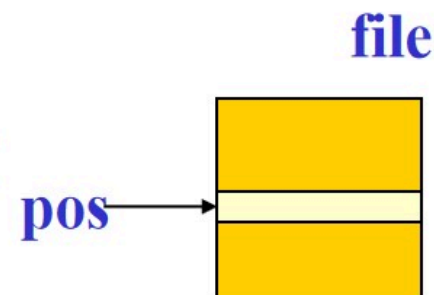
如ReadInt(), ReadLine(), WriteChar(), WriteDouble()等。

- int skipBytes(int n)：将指针乡下移动若干字节

- length()：返回文件长度

- long getFilePointer()：返回指针当前位置

- void seek(long pos)：将指针调到所需位置



4、文件操作/随机访问文件

——随机访问文件（ **RandomAccessFile** ）

在生成一个随机文件对象时，除了要指明文件对象和文件名之外，还需要指明访问文件的模式。

- **RandomAccessFile(File file, String mode)**

RandomAccessFile(String name, String mode)

- **mode** 的取值：

- “r” 只读. 任何写操作都将抛出 **IOException**。
- “rw” 读写. 文件不存在时会创建该文件，文件存在时，原文件内容不变，通过写操作改变文件内容。
- “rws” 同步读写. 等同于读写，但是任何写操作的内容都被直接写入物理文件，包括文件内容和文件属性。
- “rwd” 数据同步读写. 等同于读写，但任何内容写操作都直接写到物理文件，但对文件属性内容的修改不是这样。

4、文件操作/随机访问文件

——随机访问文件（ **RandomAccessFile** ）

```
File f = new File("file.txt");  
new RandomAccessFile(f, "r");  
new RandomAccessFile(f, "rw");  
new RandomAccessFile("file1.txt", "r");  
new RandomAccessFile("file2.txt", "rw");
```

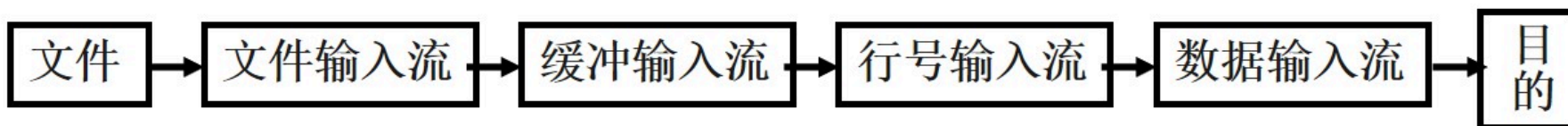
4、文件操作/随机访问文件

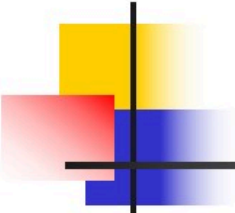
——随机访问文件（ RandomAccessFile ）

```
public class Random_file
{
    public static void main(String args[])
    {
        int data_arr[]={12, 31, 56, 23, 27, 1, 43, 65, 4, 99};
        try
        {
            RandomAccessFile randf=new RandomAccessFile("temp.dat");
            for (int i=0; i<data_arr.length; i++)
                randf.writeInt(data_arr[i]);
            for(int i=data_arr.length-1; i>=0; i--)
            {
                randf.seek(i*4L); //int数据占4个字节
                System.out.println(randf.readInt());
            }
            randf.close();
        }catch (IOException e){
            System.out.println("File access error: "+e);
        }
    }
}
```


小结

- 在Java中有数据传输的地方都用到I/O流(通常是文件、网络、内存和标准输入输出等)。
- InputStream 和OutputStream是所有字节流的祖先(只有RandomAccessFile类是一个例外), read和write是它们最基本的方法, 读写单位是字节。
- Reader 和Writer是所有字符流的祖先, read和write是它们最基本的方法, 读写单位是字符。
- 在众多的流对象中, 并不是每一种都单独使用, 其中过滤流的子类在数据送出去之前做必要的处理。





小结

- File, File(Input/Output)Stream, RandomAccessFile是处理本地文件的类。
- Data(Input/Output)Stream是一个过滤流的子类，借此可以读写各种基本数据，在文件和网络中经常使用。如：readByte, writeBoolean等。
- Buffered(Input/Output)Stream的作用是在数据送到目的之前先缓存，达到一定数量时再送到目的，以减少阻塞次数。
- Piped(Input/Output)Stream适合与一个处理的输出作为另一个处理的输入的情况。